

Lab Assignment – 4

Implementation and Analysis of Disk Scheduling Algorithms

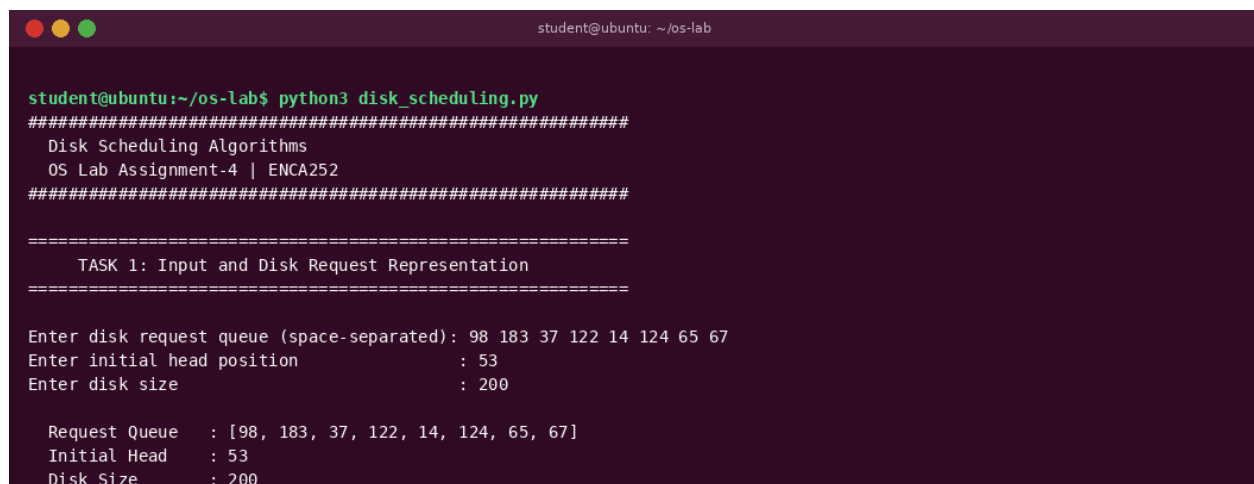
Course: Fundamentals of Operating System Lab (ENCA252)

Program: BCA (AI & DS) (Research)

Task 1: Input and Disk Request Representation

Disk request queue [98, 183, 37, 122, 14, 124, 65, 67], initial head position 53, and disk size 200 are taken as input and displayed.

```
def get_input():
    requests = list(map(int,
        input("Enter disk request queue : ").split()))
    head = int(input("Enter initial head position : "))
    disk_size = int(input("Enter disk size : "))
    print(f" Request Queue : {requests}")
    print(f" Initial Head : {head}")
    print(f" Disk Size : {disk_size}")
    return requests, head, disk_size
```



```
student@ubuntu: ~/os-lab
student@ubuntu:~/os-lab$ python3 disk_scheduling.py
#####
Disk Scheduling Algorithms
OS Lab Assignment-4 | ENCA252
#####

=====
TASK 1: Input and Disk Request Representation
=====

Enter disk request queue (space-separated): 98 183 37 122 14 124 65 67
Enter initial head position           : 53
Enter disk size                       : 200

Request Queue   : [98, 183, 37, 122, 14, 124, 65, 67]
Initial Head    : 53
Disk Size       : 200
```

Task 2: FCFS – First Come First Serve

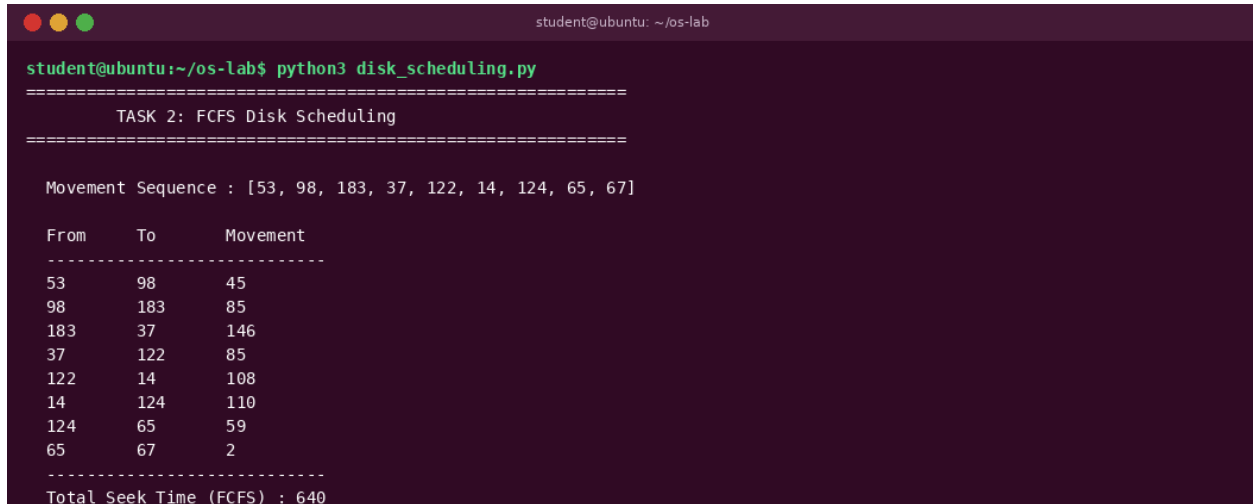
Requests are serviced in arrival order. No optimization is applied. Head moves as per the original queue sequence.

```
def fcfs(requests, head):
```

```

seek_time = 0
sequence = [head]
current = head
for req in requests:
    seek_time += abs(current - req) # head movement
    current = req
sequence.append(req)
print(f" Movement Sequence : {sequence}")
print(f" Total Seek Time (FCFS) : {seek_time}")
return seek_time

```



```

student@ubuntu: ~/os-lab
student@ubuntu:~/os-lab$ python3 disk_scheduling.py
=====
TASK 2: FCFS Disk Scheduling
=====

Movement Sequence : [53, 98, 183, 37, 122, 14, 124, 65, 67]

  From    To    Movement
-----
  53      98      45
  98     183      85
 183      37     146
  37     122      85
 122      14     108
  14     124     110
 124      65      59
  65      67       2
-----
Total Seek Time (FCFS) : 640

```

Task 3: SSTF – Shortest Seek Time First

At each step, the request closest to the current head position is selected. This greedy approach minimizes individual movements but may starve distant requests.

```

def sstf(requests, head):
    seek_time = 0
    remaining = requests.copy()
    sequence = [head]
    current = head
    while remaining:
        # pick request nearest to current head
        nearest = min(remaining, key=lambda x: abs(x - current))
        seek_time += abs(current - nearest)
        current = nearest
        sequence.append(nearest)
        remaining.remove(nearest)
    print(f" Movement Sequence : {sequence}")
    print(f" Total Seek Time (SSTF) : {seek_time}")
    return seek_time

```

```
student@ubuntu: ~/os-lab
student@ubuntu:~/os-lab$ python3 disk_scheduling.py
=====
TASK 3: SSTF Disk Scheduling
=====

Movement Sequence : [53, 65, 67, 37, 14, 98, 122, 124, 183]

  From    To    Movement
-----
  53      65      12
  65      67       2
  67      37      30
  37      14      23
  14      98      84
  98     122      24
 122     124       2
 124     183      59
-----
Total Seek Time (SSTF) : 236
```

Task 4: SCAN – Elevator Algorithm

Head moves in one direction servicing all requests, reaches the disk end, then reverses and services remaining requests on the way back.

```
def scan(requests, head, disk_size):
    seek_time = 0
    sequence = [head]
    current = head

    left = sorted([r for r in requests if r < head], reverse=True)
    right = sorted([r for r in requests if r >= head])

    for r in right: # move right
        seek_time += abs(current - r)
        current = r; sequence.append(r)
    seek_time += abs(current - (disk_size-1)) # go to end
    current = disk_size - 1
    sequence.append(disk_size - 1)

    for r in left: # reverse left
        seek_time += abs(current - r)
        current = r; sequence.append(r)
    print(f" Total Seek Time (SCAN) : {seek_time}")
    return seek_time
```

```
student@ubuntu: ~/os-lab
student@ubuntu:~/os-lab$ python3 disk_scheduling.py
=====
TASK 4: SCAN (Elevator) Algorithm
=====

Movement Sequence : [53, 65, 67, 98, 122, 124, 183, 199, 37, 14]

From    To    Movement
-----
53      65     12
65      67      2
67      98     31
98     122     24
122     124      2
124     183     59
183     199     16
199      37    162
37      14      23
-----
Total Seek Time (SCAN) : 331
```

Task 5: C-SCAN – Circular SCAN

Head moves in one direction only. After reaching the end, it jumps back to cylinder 0 and continues — providing uniform wait time for all requests.

```
def cscan(requests, head, disk_size):
    seek_time = 0
    sequence = [head]
    current = head

    left = sorted([r for r in requests if r < head])
    right = sorted([r for r in requests if r >= head])

    for r in right: # move right
        seek_time += abs(current - r)
        current = r; sequence.append(r)
    seek_time += abs(current - (disk_size-1)) # go to end
    current = disk_size - 1; sequence.append(current)
    seek_time += disk_size - 1 # wrap to 0
    current = 0; sequence.append(0)

    for r in left: # service left
        seek_time += abs(current - r)
        current = r; sequence.append(r)

    print(f" Total Seek Time (C-SCAN) : {seek_time}")
    return seek_time
```

```
student@ubuntu: ~/os-lab
student@ubuntu:~/os-lab$ python3 disk_scheduling.py
=====
TASK 5: C-SCAN Algorithm
=====

Movement Sequence : [53, 65, 67, 98, 122, 124, 183, 199, 0, 14, 37]

From    To    Movement
-----
53      65     12
65      67      2
67      98     31
98     122     24
122     124      2
124     183     59
183     199     16
199      0     199
0       14     14
14      37     23
-----
Total Seek Time (C-SCAN) : 382
```

Task 6: Performance Comparison

All four algorithms are ranked by total seek time (cylinder movements). SSTF wins with 236, followed by SCAN (331), C-SCAN (382), and FCFS (640).

```
def performance_comparison(fcfs_t, sstf_t, scan_t, cscan_t):
    results = sorted([
        ("FCFS", fcfs_t),
        ("SSTF", sstf_t),
        ("SCAN", scan_t),
        ("C-SCAN", cscan_t),
    ], key=lambda x: x[1])
    for rank, (algo, t) in enumerate(results, 1):
        print(f" {algo:<10} {t:<18} {rank}")
    print(f" Best : {results[0][0]} ({results[0][1]} cylinders)")
    print(f" Worst : {results[-1][0]} ({results[-1][1]} cylinders)")
```

```
student@ubuntu: ~/os-lab
student@ubuntu:~/os-lab$ python3 disk_scheduling.py
=====
TASK 6: Performance Comparison
=====

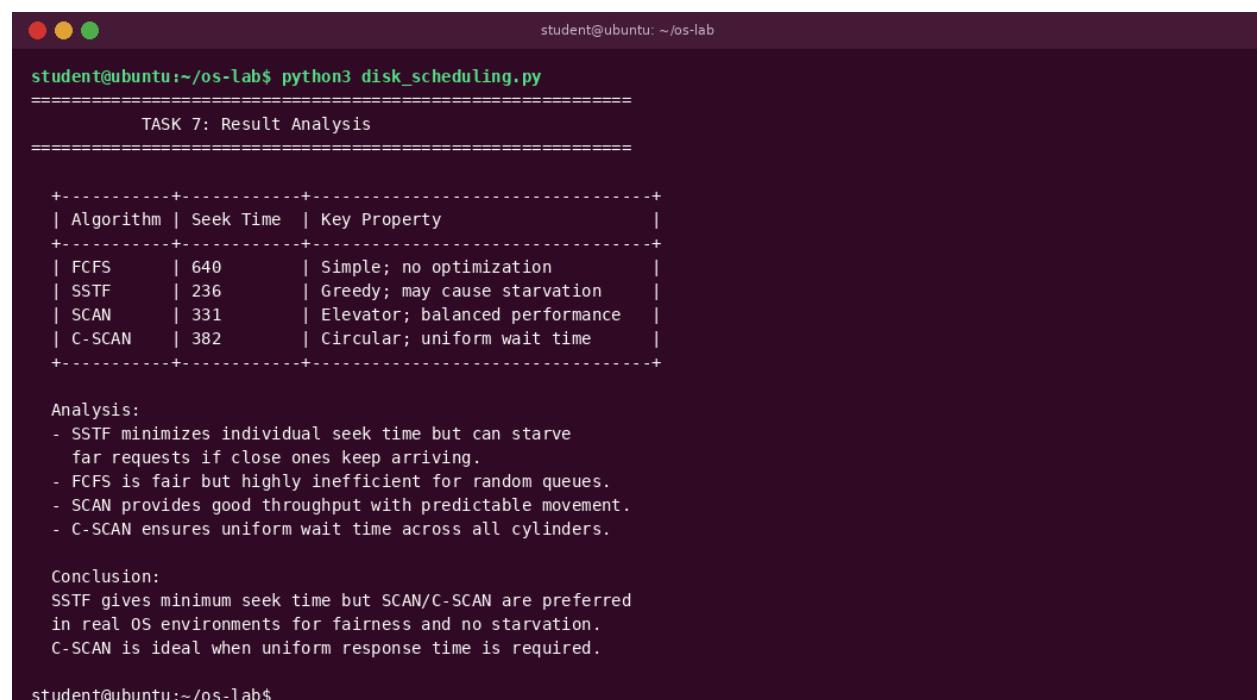
Algorithm  Total Seek Time  Rank
-----
SSTF       236              1
SCAN       331              2
C-SCAN     382              3
FCFS       640              4
-----

Best Algorithm : SSTF (236 cylinders)
Worst Algorithm : FCFS (640 cylinders)
```

Task 7: Result Analysis

SSTF gives minimum seek time but risks starvation. SCAN and C-SCAN are preferred in real systems for fairness. FCFS is simplest but most inefficient.

```
def result_analysis(fcfs_t, sstf_t, scan_t, cscan_t):
    print(f"""
+-----+-----+-----+
| Algorithm | Seek Time | Key Property |
+-----+-----+-----+
| FCFS | {fcfs_t} | Simple; no optimization |
| SSTF | {sstf_t} | Greedy; may cause starvation |
| SCAN | {scan_t} | Elevator; balanced performance |
| C-SCAN | {cscan_t} | Circular; uniform wait time |
+-----+-----+-----+
""")
```



```
student@ubuntu: ~/os-lab
student@ubuntu:~/os-lab$ python3 disk_scheduling.py
=====
TASK 7: Result Analysis
=====

+-----+-----+-----+
| Algorithm | Seek Time | Key Property |
+-----+-----+-----+
| FCFS      | 640       | Simple; no optimization |
| SSTF      | 236       | Greedy; may cause starvation |
| SCAN      | 331       | Elevator; balanced performance |
| C-SCAN    | 382       | Circular; uniform wait time |
+-----+-----+-----+

Analysis:
- SSTF minimizes individual seek time but can starve
  far requests if close ones keep arriving.
- FCFS is fair but highly inefficient for random queues.
- SCAN provides good throughput with predictable movement.
- C-SCAN ensures uniform wait time across all cylinders.

Conclusion:
SSTF gives minimum seek time but SCAN/C-SCAN are preferred
in real OS environments for fairness and no starvation.
C-SCAN is ideal when uniform response time is required.

student@ubuntu:~/os-lab$
```